

# 区块链技术详解和Python实现案例

原创 2018-03-19 Adil Moujahid [PPV课大数据](#)



区块链可以说是互联网成立以来最重要和最具颠覆性的技术之一。它是比特币和其他加密货币背后的核心技术，在过去几年引起大家广泛的关注。

区块链的核心是一个分布式数据库，允许双方直接交易，而无需中央机构，也就是通常大家所说的“去中心化”。“去中心化”这个简单而重要的概念对银行、政府和市场等机构具有重大意义，可以说，任何依赖中央数据库作为核心竞争优势的企业或组织都可能受到区块链技术的挑战甚至颠覆。

本文的目标是给你一个区块链技术的实用介绍，而不是炒作比特币和其他加密货币概念。第1节和第2节介绍了区块链一些核心概念，第3节介绍了如何使用Python实现区块链。我们还将实施2个Web前端程序，以方便用户与区块链系统交互。

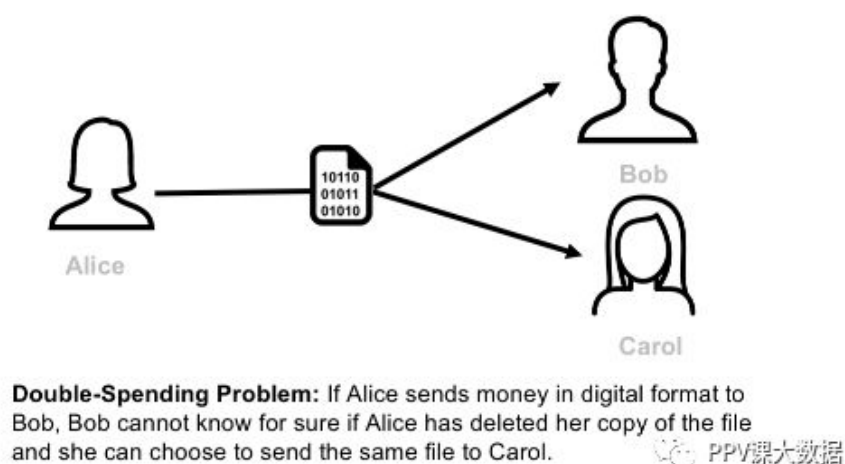
本文中，我将使用比特币作为“区块链”的例子，但本文描述的大多数概念都适用于其他加密货币。

## 1. 区块链扫盲课

2008年一个自称Satoshi Nakamoto的家伙发布了区块链白皮书，没有人知道这个Satoshi Nakamoto是谁。该白皮书的标题是“比特币：对等电子现金系统”，这个白皮书奠定了区块链的技术基础。在最初的白皮书中，Satoshi描述了如何构建一个点对点电子现金系统，使得在线支付可以直接从一方发送到另一方，而无需通过中央机构。该系统解决了数字货币中称为双重支出的许多重要问题。

### 1.1 那什么是双重支出呢？

假设Alice想支付Bob 1 美元。如果Alice和Bob使用实物现金，那么在交易执行后Alice将不再拥有1美元。如果Alice和Bob使用数字货币，那么问题变得更加复杂。数字货币是数字形式，可以很容易地复制。如果Alice通过电子邮件向Bob发送一个价值1美元的数字文件，Bob无法确定Alice是否删除了她的文件副本。如果Alice仍然拥有之前那个1美元的数字文件，那么她可以选择将同一个文件发送给Carol。这个问题叫做双重支出。



解决双重支出问题的一种方法是在Alice、Bob和网络中的所有其他参与者之间建立一个可信的第三方（例如银行）。该第三方负责集中管理账本，这个账本用于跟踪和验证网络中的所有交易。这个解决方案的缺点是要使系统正常运行，它需要信任一个集中的第三方。

## 1.2 比特币：双重支出问题的去中心化解决方案

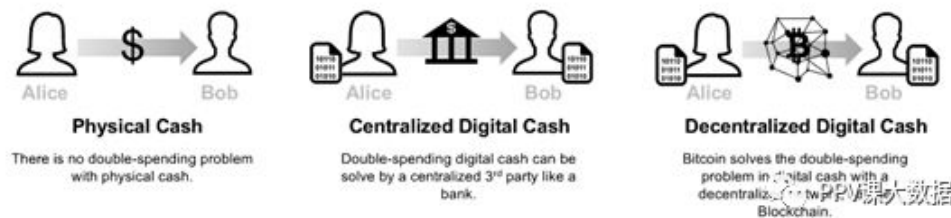
为了解决双重支出问题，Satoshi提出了一个公共账本概念，即用区块链来跟踪网络中的所有交易。区块链具有以下特点：

**分布式：**账本在多台计算机上复制，而不是存储在中央服务器上。任何具有互联网连接的计算机都可以下载区块链的完整副本；

**密码：**密码用于确保发件人拥有试图发送的比特币，并决定如何将交易添加到区块链中；

**不可变：**区块链只能以追加方式更改。换句话说，交易只能通过追加记录添加到区块链中，原有的交易记录不能被删除或修改；

**工作证明（PoW）：**那些参与破解加密算法的参与者（也被称为矿工）可以在区块链中添加交易记录。这个过程称为工作证明（Proof of Work），它会使得系统更加安全（稍后会介绍这一点）。



支付比特币的过程如下：

**步骤1（只需一次）：**创建一个比特币钱包。对于一个人发送或接收比特币，她需要创建一个比特币钱包。比特币钱包存储2条信息：私钥和公钥。私钥是一个秘密字符串，允许所有者向其他用户发送比特币，或者在用比特币付款时使用。公钥是接收比特币所需的字符串。公钥也被称为比特币地址（不完全对，但为了简单起见，我们先这么用）。请注意，钱包本身不存储比特币，比特币余额的信息是存储在比特币区块链中的；

第2步：创建一个比特币交易。如果Alice想要给Bob支付1 BTC，Alice需要使用她的私钥连接到她的比特币钱包，并创建一个包含比特币数量和发送的地址的交易信息；

第3步：将交易广播至比特币网络。一旦Alice创建比特币交易，她需要将此交易信息广播到整个比特币网络；

第4步：确认交易。一位“收听”比特币网络的矿工使用Alice的公钥验证交易，证实Alice的钱包中有足够的比特币（在这种情况下至少有1 BTC），并在比特币的区块链中增加了一个新的记录，记录其中包含交易的相关信息；

第5步：将区块链更改广播给所有矿工。一旦交易确认，矿工应向其他矿工广播区块链变更，以确保他们的区块链副本全部同步。

## 2. 区块链技术深入研究

本部分的目标是深入介绍区块链技术相关模块。我们将介绍公钥密码学、散列函数、区块链的“开采”和安全性。

### 2.1 公钥加密

公钥密码术(也称不对称密码术)：公钥可以广播，而私钥只有所有者知道。这其中包含两个功能：

(1) 认证：验证发送消息持有者是否持有与其配对的私钥；

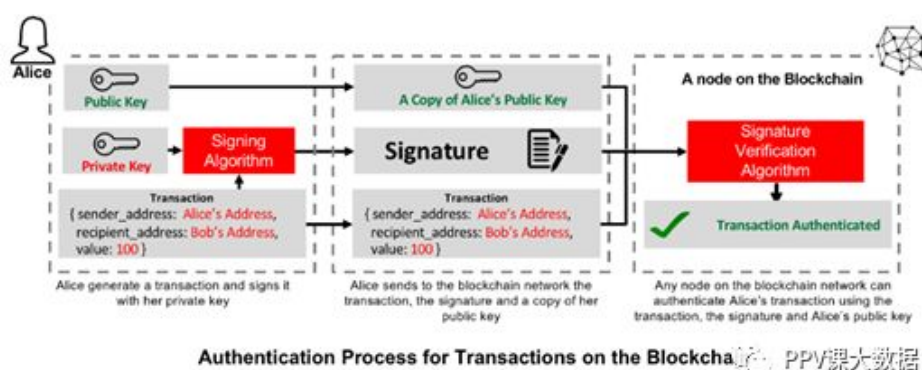
(2)加密：私钥的持有者只有用与公钥配对的私钥才能解密用公钥加密的消息。

RSA和椭圆曲线数字签名（ECDSA）是最流行的公钥密码算法。

就比特币而言，ECDSA算法用于生成比特币钱包。比特币使用各种密钥和地址，但为了简单起见，我们将在本文中假设每个比特币钱包都有一对私钥/公钥，比特币地址就是钱包的公钥。如果您对比特币钱包的完整技术细节感兴趣，我推荐这篇文章

([https://en.bitcoin.it/wiki/Technical\\_background\\_of\\_version\\_1\\_Bitcoin\\_addresses](https://en.bitcoin.it/wiki/Technical_background_of_version_1_Bitcoin_addresses))。

要发送或接收BTC，用户首先生成一个包含一对私钥和公钥的钱包。如果Alice想要给Bob发送一些BTC，她需要创建一个交易，然后在该交易中输入她和Bob的公钥，以及她想要发送的BTC数量，最后使用她的私钥签署交易。区块链中的计算机使用Alice的公钥来验证交易是否真实，并将交易记录添加到区块链的区块。



## 2.2 哈希函数和“挖矿”

所有比特币交易都分别记录在称为块的文件中。比特币每10分钟添加一个新的交易块。一旦新块被添加到区块链中，它就不能被更改，也不能被删除。矿工负责创建新的交易块。矿工必须使用发送者的公钥验证每笔交易，确认发送者有足够的余额用于此次交易请求，确认通过后交易将添加到区块中。矿工们可以自由地选择在区块中添加哪些交易，因此发送者需要给矿工支付交易费以激励矿工将其交易记录添加到区块中。

区块加入区块链的过程，被称之为“挖矿”，“挖矿”的参与者被称之为矿工，矿工需要破解一个加密问题才能讲区块加入区块链。成功破解加密问题的矿工将获得比特币奖励(注意这和上面的交易费是不同的)。“挖矿”过程也被称为工作量证明（PoW），它是使区块链变得安全和可靠的主要机制（安全性问题稍后会详述）。

## 哈希和区块链的密码拼图

要理解区块链的密码拼图，我们需要从哈希函数开始。哈希函数可用于将任意大小的数据映射到固定大小的数据。哈希函数返回的值称为散列值，哈希函数通常用于通过检测重复记录来加速数据库查找，它也广泛用于密码学。密码哈希函数可以验证某些输入数据和给定的散列值之间的映射关系，但如果输入数据是未知的，则要想通过散列值反推出输入数据是非常困难的。 [2]

比特币使用称为SHA-256的哈希加密算法，SHA-256应用于块数据（比特币交易）和一个称为nonce的随机数组合，通过更改块数据或随机数，我们可以得到完全不同的散列值。一个对被认为有效(已“开采”)的块，它的散列值和随机数需要满足一定的条件，例如，散列值的前4位数字需要为“0000”。我们可以通过使条件更复杂来增加“挖矿”的复杂性，例如我们可以增加散列值开始所需的0的数量。

矿工需要找到一个随机数值，使得散列值满足“开采”条件。可以使用下面的应用程序来模拟这个过程，当你在文本框输入数据或更改随机数值时，你可以注意到哈希值的变化。当你点击“挖矿”按钮时，应用程序nonce从0开始，计算散列值并检查散列值的前四位数是否等于“0000”。如果前四位数字不等于“0000”，它将随机数加1，并重复整个过程直到找到一个满足条件的随机数值。

Nonce: 72608

Data:

Hash: b8f178d5b3b8482247aab03562683980b93ac32fe4a0e3cc7d0e9dab84a7f471

Mine

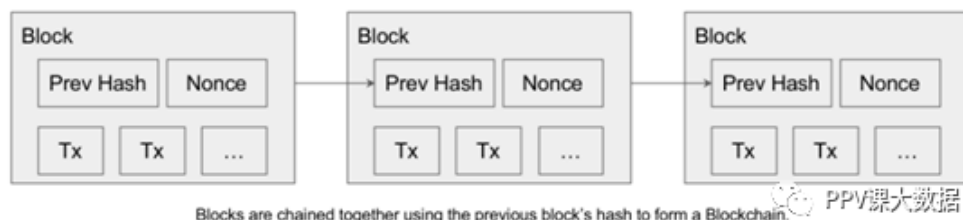
PPV课大数据

(译者注：此图仅做静态展示，程序演示请访问原文连接)

## 2.3从块到区块链

正如前面部分所讨论的，交易以区块形式分组，并且块被附加到区块链。为了创建区块链，每个新块都使用前一个块的哈希散列作为其数据的一部分。为了创建一个新块，矿工会在“挖矿”过程中添加上一个块的散列并以2.2中说述的方式“开采”有效的区块。

任何区块中的数据变更都会影响到它后面区块的散列值，每一个区块都会使用前一个块的哈希散列作为其数据的一部分，如果前一个区块数据变了，后面的区块将会成为“无效”的区块，区块链也因此具有了不可变更的特性。



你可以使用下面的应用程序来模拟有3个区块的区块链。当你输入“Data”文本框或更改nonce值时，可以注意到下一个块的散列值和“Prev”值（前一个散列）的更改。您可以通过单击每个块的“开采”按钮来模拟采矿过程。在挖出3个区块之后，尝试更改块1或块2中的数据，并且您会注意到后面的所有块都变为无效。（译者注：程序演示请访问原文链接）

## 2.4 将区块添加到区块链

比特币网络中的所有矿工彼此竞争，看谁能先找到有效的区块，先找到的可以将其添加到区块链中，并从网络中获得回报。发现一个有效的随机数(上文中那个nonce值)是非常困难的，但正



所谓重赏之下必有勇夫，总有人会尝试并挖到矿。那如果两名矿工或更多的人同时递交他们的区块，会发生什么？

## 冲突解决

如果两个矿工几乎同时解决一个块，那么我们将在网络中有两个不同的块链(区块链1和区块链2)，我们需要等待下一个区块来解决冲突问题。一些矿工将继续尝试在区块链1和区块链2顶部开采，如果新区块在区块链1的顶部被开采，则区块链2变为无效，开采区块链1的矿工因此获得奖励，与此同时区块链2的那部分未被添加到区块链的交易信息将返回到事务池，并添加到下一个块。简而言之，如果区块链上存在冲突，那么长一点的那个链将会是赢家。

## 2.5 区块链和双重支出

在本节中，我们将介绍区块链对双重支出攻击的常见处理方式，以及用户应采取哪些措施来防止他们遭受损失。

### 种族攻击

攻击者快速连续发送同一个币到两个不同的地址。为了防止这种攻击，只要在接受付款之前等待至少一个数据块确认就可以避免。 [3]

### 芬尼攻击

攻击者预先在事务中“开采”一个块，并在释放块之前在第二个事务中花费相同的币。在这种情况下，第二个事务将不会被验证。为了防止这种攻击，只要在接受付款之前等待至少6个数据块确认(?)就可以避免。 [3]



## 多数人攻击（也被称为51%攻击）

在这次攻击中，攻击者拥有51%的网络计算能力。攻击者首先在整个网络上创建一个交易，然后“开采”一个私人区块链，在这私人区块链里他将交易信息篡改，如收到100币变成收到200币。由于攻击者拥有大部分计算能力，因此他可以保证他在交易的这个时间点的链路比“诚实的”区块链更长，根据区块链规则，这些私有的区块将成为“诚实的”区块，而这笔篡改的交易也披上了“合法”的外衣。当然，这种攻击的可能性非常小，因为获得51%的计算能力成本太高了，几乎没有人负担的起。 [4]

## 3. Python中的区块链实现

在本节中，我们将使用Python实现基本区块链和区块链客户端。

我们的区块链将具有以下功能：

- 1) 将多个节点添加到区块链
- 2) 工作证明（PoW）
- 3) 节点之间的冲突解决
- 4) 使用RSA加密交易

我们的区块链客户端将具有以下功能：

- 1) 使用公钥/私钥加密（基于RSA算法）生成钱包
- 2) 使用RSA加密生成事务

我们还将实施2个仪表板：

1) 面向矿工的“区块链前台”

2) “区块链客户端”让用户生成钱包并发送硬币

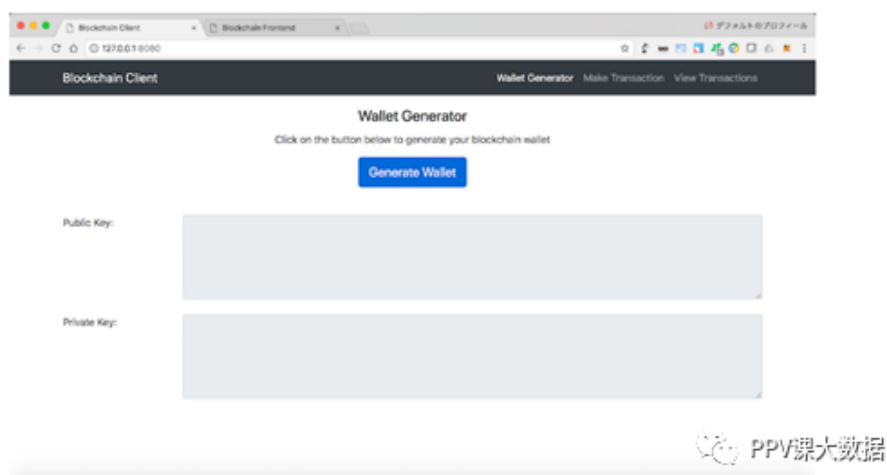
本项目基于这个github项目(<https://github.com/dvf/blockchain>)。我对原代码进行了一些修改，目的是为事务添加RSA加密。钱包生成和交易加密基于Jupyter编辑器，2个仪表板使用HTML / CSS / JS实现。

你可以从<https://github.com/adilmoujahid/blockchain-python-tutorial>下载完整的源代码。

请注意，此项目仅用于学习，不能应用于生产，它安全性不够好，也缺少许多重要功能。

### 3.1 区块链客户端实施

你可以通过访问blockchain\_client文件夹并键入`python blockchain_client.py`来从终端启动区块链客户端。在浏览器中输入`http://localhost:8080`，你将看到下面的仪表板：



仪表板在导航栏中有3个选项卡：

1) 钱包生成器：使用RSA加密算法生成钱包（公钥/私钥对）；

2) 进行交易：生成交易并将其发送到区块链节点；

3) 查看转换：查看区块链上的交易。为了制作或查看交易，至少需要运行一个区块链节点（将在下一节中介绍）。

以下是blockchain\_client.py代码中最重要部分的一些说明。我们定义了一个名为Transaction的python类，它具有4个属性sender\_address, sender\_private\_key, recipient\_address, value。 这些是发件人创建交易所需的4条信息。

to\_dict（）方法以Python字典格式（没有发件人的私钥）返回交易信息。

sign\_transaction（）方法获取交易信息（不包括发件人的私钥），并使用发件人的私钥对其进行签名。

```
class Transaction:

    def __init__(self, sender_address, sender_private_key, recipient_address, value):
        self.sender_address = sender_address
        self.sender_private_key = sender_private_key
        self.recipient_address = recipient_address
        self.value = value

    def __getattr__(self, attr):
        return self.data[attr]

    def to_dict(self):
        return OrderedDict({'sender_address': self.sender_address,
                            'recipient_address': self.recipient_address,
                            'value': self.value})

    def sign_transaction(self):
        """
        Sign transaction with private key
        """
        private_key = RSA.importKey(binascii.unhexlify(self.sender_private_key))
        signer = PKCS1_v1_5.new(private_key)
        h = SHA.new(str(self.to_dict()).encode('utf8'))
        return binascii.hexlify(signer.sign(h)).decode('ascii')
```



下面的代码启动一个Python Flask应用程序，我们将使用它来创建不同的API来与区块链及其客户端进行交互。

```
app = Flask(__name__)
```

下面我们定义返回html页面的3个Flask路由，每个标签页都有一个html页面。

```

@app.route('/')
def index():
    return render_template('./index.html')

@app.route('/make/transaction')
def make_transaction():
    return render_template('./make_transaction.html')

@app.route('/view/transactions')
def view_transaction():
    return render_template('./view_transactions.html')

```

PPV课大数据

下面我们定义一个生成钱包的API（私钥/公钥对）。

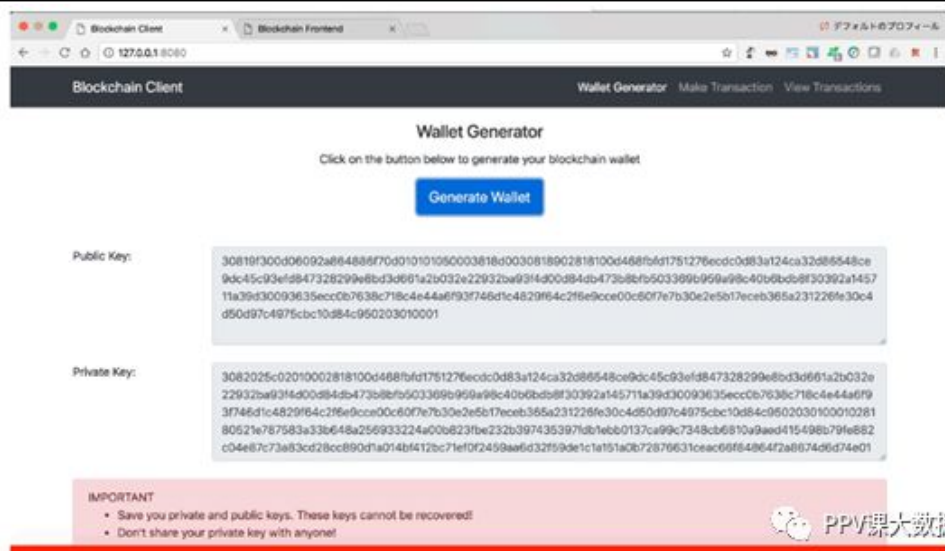
```

@app.route('/wallet/new', methods=['GET'])
def new_wallet():
    random_gen = Crypto.Random.new().read
    private_key = RSA.generate(1024, random_gen)
    public_key = private_key.publickey()
    response = {
        'private_key': binascii.hexlify(private_key.exportKey(format='DER')).decode('ascii'),
        'public_key': binascii.hexlify(public_key.exportKey(format='DER')).decode('ascii')
    }

    return jsonify(response), 200

```

PPV课大数据



PPV课大数据

下面我们定义一个API，它将sender\_address, sender\_private\_key, recipient\_address, value作为输入值，并返回事务（无私钥）和签名。

```

@app.route('/generate/transaction', methods=['POST'])
def generate_transaction():
    sender_address = request.form['sender_address']
    sender_private_key = request.form['sender_private_key']
    recipient_address = request.form['recipient_address']
    value = request.form['amount']

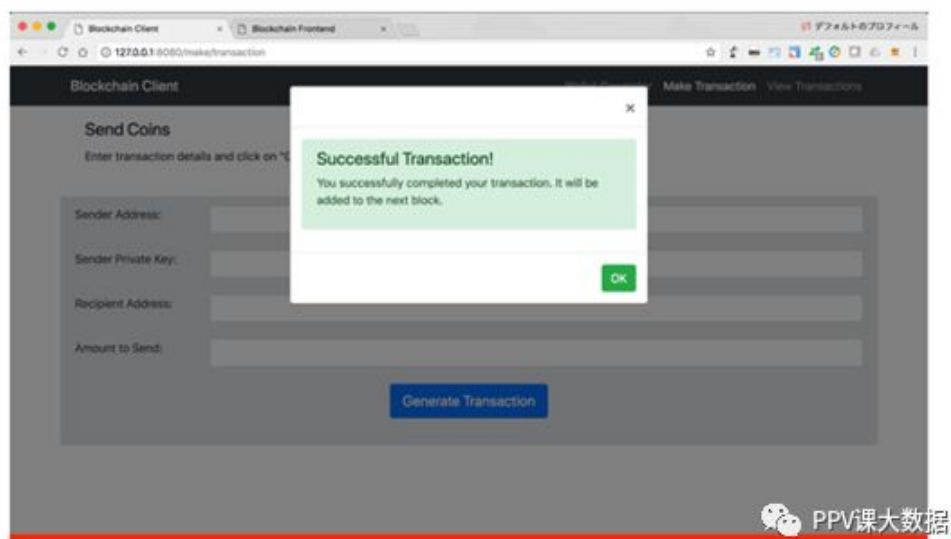
    transaction = Transaction(sender_address, sender_private_key, recipient_address, value)

    response = {'transaction': transaction.to_dict(), 'signature': transaction.sign_transaction()}

    return jsonify(response), 200

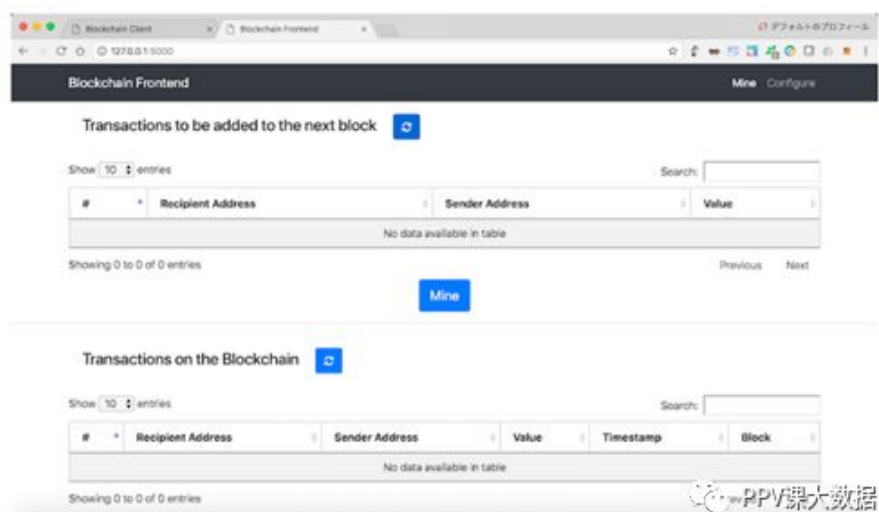
```

PPV课大数据



### 3. 2区块链实施

你可以在blockchain文件夹启动区块链节点，然后输入`python blockchain_client.py`或`python blockchain_client.py -p <端口号>`。如果未指定端口号，则它将默认为端口5000. 在浏览器中，转至`http: // localhost: <PORT NUMBER>`以查看区块链前端仪表板。



仪表板在导航栏中有2个选项卡：

1. 挖矿：用于查看交易和区块链数据，以及用于挖掘新的交易区块；
2. 配置：用于配置不同区块链节点之间的连接；

以下是blockchain.py代码中最重要部分的一些说明。

我们首先定义一个具有以下属性的区块链类：

交易：将被添加交易列表；

链：一个有数据构建的区块链；

节点：包含节点URL的集合。区块链使用这些节点从其他节点检索区块链数据，并在区块链不同步时更新区块链；

node\_id：识别区块链节点的随机字符串；

Blockchain类还实现了以下方法：

register\_node (node\_url)：将新的区块链节点添加到节点列表中

verify\_transaction\_signature (sender\_address, signature, transaction)：检查提供的签名是否与由公钥签名的事务 (sender\_address) 相对应；

submit\_transaction (sender\_address, recipient\_address, value, signature)：如果验证签名，则将事务添加到事务列表；

create\_block (nonce, previous\_hash)：向区块链添加一个交易块；

散列（块）：创建块的SHA-256散列；

proof\_of\_work ()：工作证明算法。寻找满足采矿条件的随机数；

valid\_proof (transactions, last\_hash, nonce, difficulty = MINING\_DIFFICULTY)：检查哈希值是否满足挖掘条件。该函数在proof\_of\_work函数中使用；

`valid_chain (链)`：检查链接是否有效；

`resolve_conflicts ()`：通过替换网络中最长的链来解决区块链节点之间的冲突；

下面的代码启动一个Python Flask应用程序，我们将使用它来创建不同的API来与区块链进行交互。`proof_of_work ()`：工作证明算法。寻找满足采矿条件的随机数。

`valid_proof (transactions, last_hash, nonce, difficulty = MINING_DIFFICULTY)`：检查哈希值是否满足挖掘条件。该函数在`proof_of_work`函数中使用。

`valid_chain (链)`：检查链接是否有效。

`resolve_conflicts ()`：通过替换网络中最长的链来解决区块链节点之间的冲突。

```
class Blockchain:

    def __init__(self):

        self.transactions = []
        self.chain = []
        self.nodes = set()
        #Generate random number to be used as node_id
        self.node_id = str(uuid4()).replace('-', '')
        #Create genesis block
        self.create_block(0, '00')

    def register_node(self, node_url):
        """
        Add a new node to the list of nodes
        """
        ...

    def verify_transaction_signature(self, sender_address, signature, transaction):
        """
        Check that the provided signature corresponds to transaction
        signed by the public key (sender_address)
        """
        ...

    def submit_transaction(self, sender_address, recipient_address, value, signature):
        """
        Add a transaction to transactions array if the signature verified
        """
        ...

    def create_block(self, nonce, previous_hash):
        """
        Add a block of transactions to the blockchain
        """
        ...
```



```

def hash(self, block):
    """
    Create a SHA-256 hash of a block
    """
    ...

def proof_of_work(self):
    """
    Proof of work algorithm
    """
    ...

def valid_proof(self, transactions, last_hash, nonce, difficulty=MINING_DIFFICULTY):
    """
    Check if a hash value satisfies the mining conditions. This function is used within the proof_of_work function.
    """
    ...

def valid_chain(self, chain):
    """
    check if a bockchain is valid
    """
    ...

def resolve_conflicts(self):
    """
    Resolve conflicts between blockchain's nodes
    by replacing our chain with the longest one in the network.
    """
    ...

```



下面的代码启动一个Python Flask应用程序，我们将使用它来创建不同的API来与区块链进行交互。

```

app = Flask(__name__)
CORS(app)

```



接下来，我们启动一个Blockchain实例。

```

blockchain = Blockchain()

```

下面我们定义返回我们区块链前端仪表盘html页面的2个Flask路由。

```

@app.route('/')
def index():
    return render_template('./index.html')

@app.route('/configure')
def configure():
    return render_template('./configure.html')

```



下面我们定义Flask API来管理交易和挖掘区块链。

'/ transactions / new': 这个API作

为'sender\_address', 'recipient\_address', 'amount'和'signature'的输入，并且将交易添加到交易列表中，如果签名有效，交易列表将被添加到下一个块中。

'/ transactions / get': 该API返回将被添加到下一个块的所有事务。

'/ chain': 该API返回所有区块链数据。

'/ mine': 该API运行工作证明算法，并将新的交易块添加到区块链。

```
@app.route('/transactions/new', methods=['POST'])
def new_transaction():
    values = request.form

    # Check that the required fields are in the POST'ed data
    required = ['sender_address', 'recipient_address', 'amount', 'signature']
    if not all(k in values for k in required):
        return 'Missing values', 400
    # Create a new Transaction
    transaction_result = blockchain.submit_transaction(values['sender_address'], values['recipient_address'],
    values['amount'], values['signature'])

    if transaction_result == False:
        response = {'message': 'Invalid Transaction!'}
        return jsonify(response), 406
    else:
        response = {'message': 'Transaction will be added to Block ' + str(transaction_result)}
        return jsonify(response), 201

@app.route('/transactions/get', methods=['GET'])
def get_transactions():
    #Get transactions from transactions pool
    transactions = blockchain.transactions

    response = {'transactions': transactions}
    return jsonify(response), 200

@app.route('/chain', methods=['GET'])
def full_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain),
    }
    return jsonify(response), 200
```

PPV课大数据

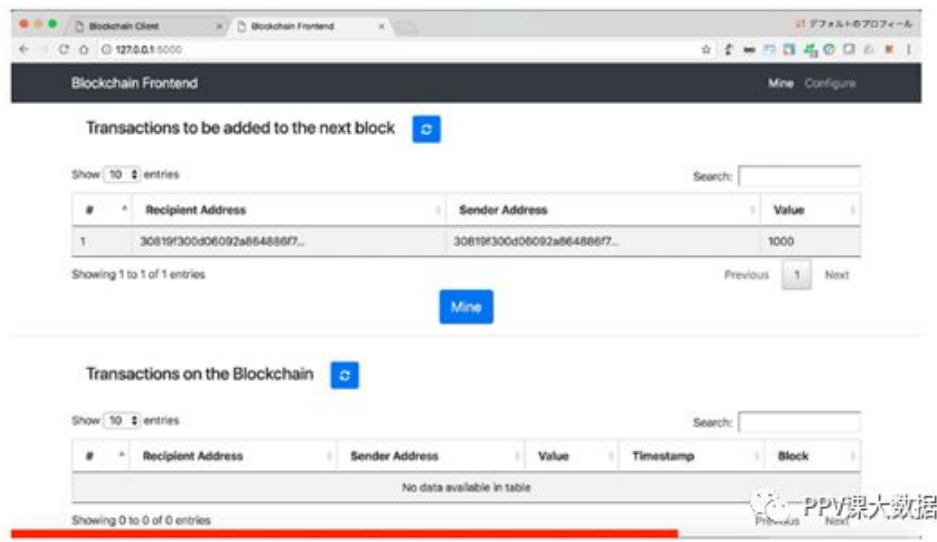
```
@app.route('/mine', methods=['GET'])
def mine():
    # We run the proof of work algorithm to get the next proof...
    last_block = blockchain.chain[-1]
    nonce = blockchain.proof_of_work()

    # We must receive a reward for finding the proof.
    blockchain.submit_transaction(sender_address=MINING_SENDER, recipient_address=blockchain.node_id, value=MINING_REWARD, si
gnature='')

    # Forge the new Block by adding it to the chain
    previous_hash = blockchain.hash(last_block)
    block = blockchain.create_block(nonce, previous_hash)

    response = {
        'message': "New Block Forged",
        'block_number': block['block_number'],
        'transactions': block['transactions'],
        'nonce': block['nonce'],
        'previous_hash': block['previous_hash'],
    }
    return jsonify(response), 200
```

PPV课大数据



下面我们定义Flask API来管理区块链节点。

’ / nodes / register ’：该API将节点URL列表作为输入，并将它们添加到节点列表中。

’ / nodes / resolve ’：该API通过用网络中可用的最长链替换本地链来解决区块链节点之间的冲突。

’ / nodes / get ’：该API返回节点列表。

```

@app.route('/nodes/register', methods=['POST'])
def register_nodes():
    values = request.form
    nodes = values.get('nodes').replace(" ", "").split(',')

    if nodes is None:
        return "Error: Please supply a valid list of nodes", 400

    for node in nodes:
        blockchain.register_node(node)

    response = {
        'message': 'New nodes have been added',
        'total_nodes': [node for node in blockchain.nodes],
    }
    return jsonify(response), 201

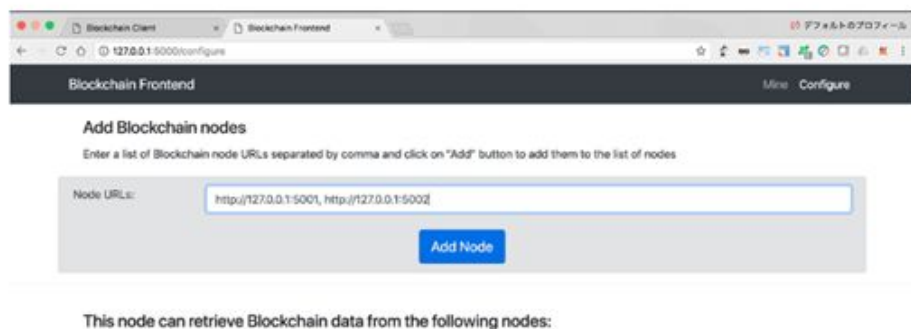
@app.route('/nodes/resolve', methods=['GET'])
def consensus():
    replaced = blockchain.resolve_conflicts()

    if replaced:
        response = {
            'message': 'Our chain was replaced',
            'new_chain': blockchain.chain
        }
    else:
        response = {
            'message': 'Our chain is authoritative',
            'chain': blockchain.chain
        }
    return jsonify(response), 200

@app.route('/nodes/get', methods=['GET'])
def get_nodes():
    nodes = list(blockchain.nodes)
    response = {'nodes': nodes}
    return jsonify(response), 200

```

PPV课大数据



PPV课大数据

下面的代码启动一个Python Flask应用程序，我们将使用它来创建不同的API来与区块链进行交互。

接下来，我们启动一个Blockchain实例。

下面我们定义返回我们区块链前端仪表盘html页面的2个Flask路由。

下面我们定义Flask API来管理交易和挖掘区块链。

`'/ transactions / new'`：这个API作

为`'sender_address'`，`'recipient_address'`，`'amount'`和`'signature'`的输入，并且将交易添加到交易列表中，如果签名有效，交易列表将被添加到下一个块中。

`'/ transactions / get'`：该API返回将被添加到下一个块的所有事务。

`'/ chain'`：该API返回所有区块链数据。

`'/ mine'`：该API运行工作证明算法，并将新的交易块添加到区块链。

下面我们定义Flask API来管理区块链节点。

`'/ nodes / register'`：该API将节点URL列表作为输入，并将它们添加到节点列表中。

`'/ nodes / resolve'`：该API通过用网络中可用的最长链替换本地链来解决区块链节点之间的冲突。

`'/ nodes / get'`：该API返回节点列表。

## 4、结论

在本文中，我们介绍了区块链的一些核心概念，并学习了如何使用Python实现一个实例。为了简单起见，我没有介绍一些技术细节，例如：钱包地址和默克尔树。如果你想了解更多建议阅读比特币白皮书。

## 参考

1 - 维基百科 - 公钥密码学

2 - 维基百科 - 哈希函数

3 - Bitcoin Stackexchange - 交易一旦产生会发生什么？

4 - 比特币维基 - 多数人攻击

END.

原文连接：

[http://adilmoujahid.com/posts/2018/03/intro-blockchain-bitcoin-python/?utm\\_source=wanqu.co&utm\\_campaign=Wanqu+Daily&utm\\_medium=website](http://adilmoujahid.com/posts/2018/03/intro-blockchain-bitcoin-python/?utm_source=wanqu.co&utm_campaign=Wanqu+Daily&utm_medium=website)

PPV课翻译作品

翻译：数据小咖



最专业的数据科学社区

前沿知识 | 免费资料 | 职场指南

数据分析 | 大数据 | 人工智能



长按/扫码关注我们